AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2025

Project:
Face Recognition System

Nguyễn Đình Tiềm Nguyễn Đăng Nhã
Xỉn Quý Hùng Đinh Quang Vinh

I. Giới thiệu

Trong lĩnh vực Computer Vision, bài toán Face Recognition (nhận diện khuôn mặt) là một trong những bài toán phổ biến nhờ vào việc có tính ứng dụng cao trong thực tiễn. Bài toán yêu cầu hệ thống có khả năng xác định danh tính của một người dựa trên hình ảnh khuôn mặt, thông qua việc so sánh khuôn mặt trong ảnh đầu vào với các khuôn mặt đã được lưu trữ trong cơ sở dữ liệu.

Trong cuộc sống, ta thường bắt gặp các hệ thống nhận diện khuôn mặt trong nhiều ứng dụng khác nhau như mở khóa điện thoại, hệ thống kiểm soát ra vào, chấm công hay xác thực danh tính trong các dịch vụ trực tuyến.

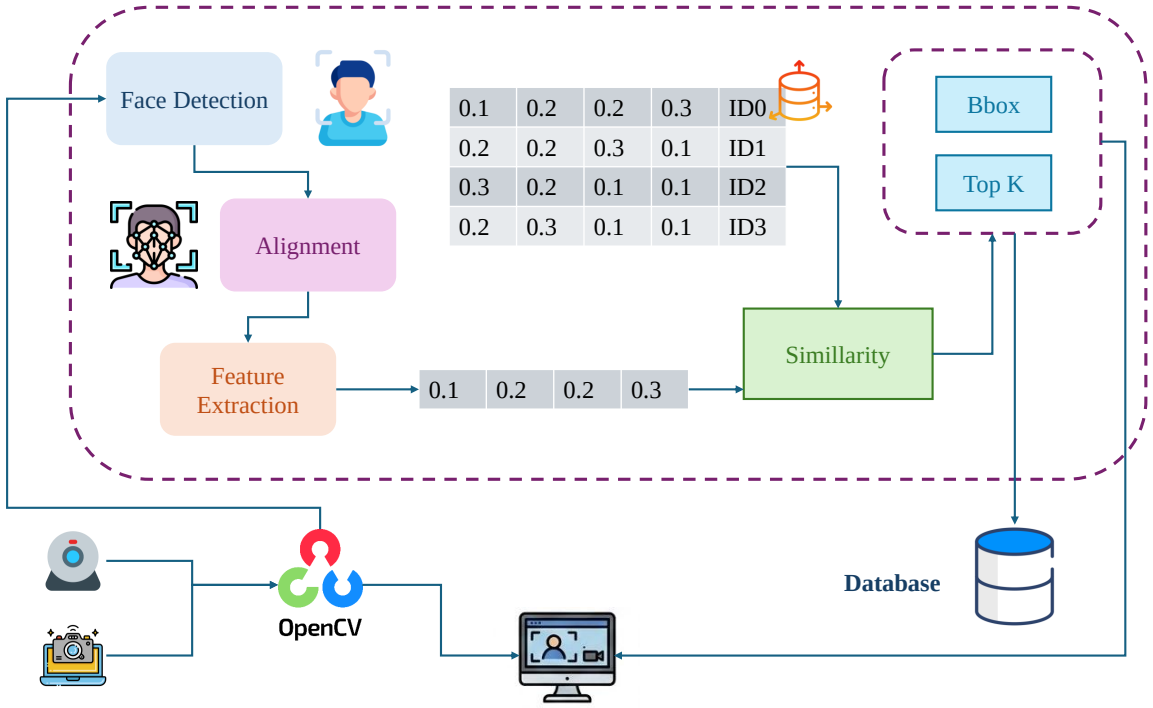


Hình 1: Minh họa cho ứng dụng của bài toán Face Recognition

Tuy nhiên, việc phát triển một hệ thống Face Recognition hoạt động tốt trong thực tiễn không phải là việc dễ dàng. Các vấn đề về dữ liệu như pose (góc nhìn của khuôn mặt), illumination (điều kiện ánh sáng) hay expression (biểu cảm khuôn mặt) đòi hỏi mô hình phải có tính tổng quát cao để có thể nhận diện chính xác trong nhiều điều kiện khác nhau. Không chỉ vậy, hệ thống không chỉ cần phải có độ chính xác cao mà còn phải nhanh, chạy được trong thời gian thực để có thể ứng dụng trong các hệ thống thực tế.

Trong những năm gần đây, nhờ vào sự bùng nổ của Deep Learning với các mô hình detection và feature extraction, bài toán này đã đạt được nhiều bước tiến lớn. Hiện nay, pipeline phổ biến của bài toán thường bao gồm các bước: Face Detection (phát hiện vị trí khuôn mặt trong ảnh),

Face Alignment (căn chỉnh khuôn mặt về cùng một chuẩn hình học), Feature Extraction (trích xuất đặc trưng khuôn mặt) và Feature Matching (so khớp đặc trưng để xác định danh tính).



Hình 2: Tổng quan về pipeline của dự án

Trong dự án này, chúng ta sẽ xây dựng một hệ thống Face Recognition hoàn chỉnh dựa trên pipeline trên. Hệ thống sẽ có giao diện hoàn chỉnh, cho phép người dùng truy cập camera để thực hiện nhận diện khuôn mặt theo thời gian thực, đồng thời hỗ trợ việc đăng ký (enroll) và nhận diện danh tính người dùng. Dự án sẽ bao gồm các bước chính sau:

1. Xây dựng mô hình Face Detection với SCRFD. Mô hình này sẽ có nhiệm vụ phát hiện và trích xuất khuôn mặt từ dữ liệu đầu vào.

2. Xây dựng mô hình Feature Extraction với ArcFace. Mô hình này sẽ tạo ra các vector đặc trưng biểu diễn khuôn mặt trong không gian đặc trưng, từ đó cho phép so sánh và xác định mức độ tương đồng giữa các khuôn mặt khác nhau.

Cuối cùng, dựa trên các mô hình đã được xây dựng, ta sẽ xây dựng một hệ thống Face Recognition hoàn chỉnh, tích hợp đầy đủ các thành phần trong pipeline để thực hiện nhận diện khuôn mặt theo thời gian thực. Hệ thống sẽ cho phép đăng ký khuôn mặt người dùng, lưu trữ đặc trưng và thực hiện so khớp để xác định danh tính khi có khuôn mặt xuất hiện trước camera.

Mục lục

I. Giới thiệu **1**

II. Face Detection **4**

II.1. Giới thiệu về Face Detection 4

II.2. Mô hình SCRFD 5

II.3. Chuẩn bị dữ liệu 6

II.4. Huấn luyện mô hình 14

II.5. Face Alignment 14

III. Feature Extraction **16**

III.1. Giới thiệu về Feature Extraction 16

III.2. Phương pháp ArcFace 17

IV. Feature Matching **18**

V. Triển khai hệ thống **19**

V.1. Tổng quan về luồng hoạt động 19

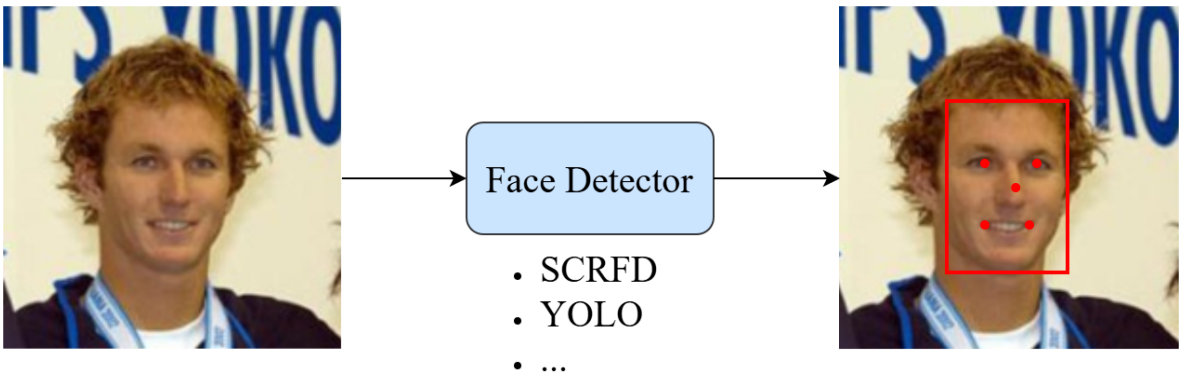
V.2. Xử lý camera 20

Phụ lục **22**

II. Face Detection

II.1. Giới thiệu về Face Detection

Face Detection (phát hiện khuôn mặt) là bước đầu tiên trong bài toán Face Recognition, có vai trò rất lớn trong việc nâng cao độ chính xác của toàn bộ hệ thống. Ở bước này, ta sẽ trích xuất vùng chứa vùng chứa khuôn mặt từ tấm ảnh đầu vào, thường được biểu diễn bằng một hộp bao (bounding box) quanh khuôn mặt đó, kèm theo các keypoints thể hiện những đặc trưng quan trọng trên khuôn mặt như hai mắt, mũi và hai khóe miệng.



Hình 3: Minh họa đầu vào và đầu ra bước Face Detection

Cụ thể, bài toán có đầu vào và đầu ra như sau:

- **Input:** Ảnh đầu vào có kích thước $x \in \mathbb{R}^{C \times H \times W}$
- **Output:** Tập các bounding boxes $B = \{b_i\}_{i=1}^{N}$ và các keypoints $K_i = \{(x_{i,j}, y_{i,j})\}_{j=1}^{M}$.

Trong đó:

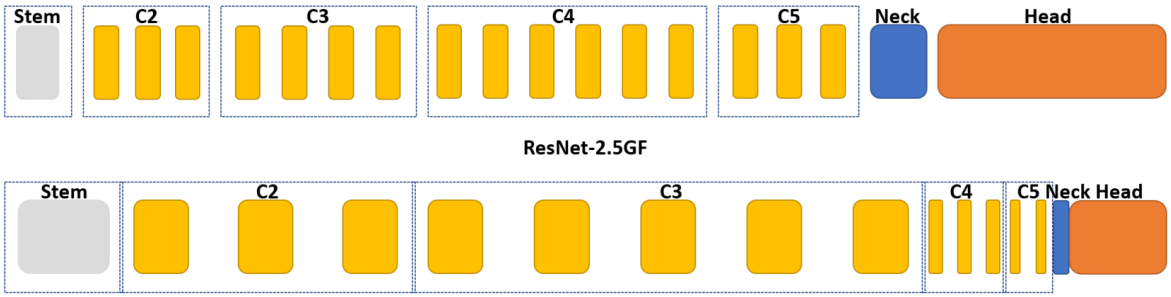
- C, H và W lần lượt là số kênh màu, chiều cao và chiều rộng của ảnh.
- N là số lượng khuôn mặt được phát hiện trong ảnh.
- b_i là bounding box của khuôn mặt thứ i. Bounding box có thể được biểu diễn dưới hai dạng phổ biến: dạng *corner* $b_i = (x_1, y_1, x_2, y_2)$ với (x_1, y_1) và (x_2, y_2) lần lượt là tọa độ góc trên bên trái và góc dưới bên phải, hoặc dạng *center* $b_i = (x, y, w, h)$ với (x, y) là tọa độ tâm của bounding box, còn w và h lần lượt là chiều rộng và chiều cao.
- $(x_{i,j}, y_{i,j})$ là tọa độ của keypoint thứ j trên khuôn mặt thứ i

Để thực hiện bước này, ta sẽ sử dụng các mô hình Deep Learning có khả năng phát hiện khuôn mặt với độ chính xác cao cũng như tốc độ suy luận nhanh để đáp ứng yêu cầu thời gian thực. Một số mô hình phổ biến có thể được sử dụng cho bài toán này bao gồm SCRFD hoặc các mô hình thuộc họ YOLO.

II.2. Mô hình SCRFD

Các mô hình Face Detection có độ phức tạp cao tuy có thể đạt được độ chính xác tốt, nhưng thường đi kèm với chi phí tính toán lớn, dẫn đến tốc độ xử lý không đáp ứng được các ứng dụng trong thời gian thực. Do đó, một thách thức lớn của bài toán là phải thiết kế hoặc lựa chọn mô hình có thể cân bằng giữa độ chính xác và tốc độ suy luận, nhằm đảm bảo hệ thống hoạt động hiệu quả trong các ứng dụng thực tế.

Sample and **C**omputation **R**edistribution for Efficient **F**ace **D**etection (SCRFD) tiếp cận vấn đề này bằng cách phân bổ lại tài nguyên tính toán và mẫu huấn luyện một cách hợp lý trong quá trình huấn luyện mô hình. Thay vì chỉ tập trung vào việc tăng độ phức tạp của kiến trúc mạng, SCRFD hướng đến việc tối ưu hóa hiệu quả sử dụng tài nguyên để đạt được sự cân bằng tốt hơn giữa độ chính xác và tốc độ suy luận.



Hình 4: Minh họa cơ chế phân bổ lại tài nguyên tính toán của SCRFD. So với kiến trúc ResNet ban đầu, SCRFD phân phối lại độ phức tạp tính toán giữa các stage của backbone và phần detection head, từ đó cải thiện sự cân bằng giữa độ chính xác và tốc độ suy luận.

Cụ thể, hai cách tối ưu chính của SCRFD bao gồm:

- **Sample Redistribution:** Phân bổ lại các mẫu huấn luyện giữa các mức đặc trưng khác nhau trong mạng (feature pyramid) nhằm đảm bảo mỗi tầng nhận được số lượng mẫu phù hợp với khả năng biểu diễn của nó. Điều này giúp cải thiện hiệu quả học của mô hình và tăng độ chính xác trong việc phát hiện khuôn mặt ở nhiều kích thước khác nhau.
- **Computation Redistribution:** Phân bổ lại tài nguyên tính toán trong kiến trúc mạng, tập trung nhiều hơn vào các tầng đặc trưng quan trọng cho việc phát hiện khuôn mặt. Cách tiếp cận này giúp tối ưu hóa việc sử dụng tài nguyên tính toán, từ đó đạt được sự cân bằng tốt hơn giữa độ chính xác và tốc độ suy luận.

Nhờ đó, mô hình có thể đạt được hiệu năng cao trong khi vẫn đảm bảo khả năng chạy trong thời gian thực trên các thiết bị có tài nguyên tính toán hạn chế.

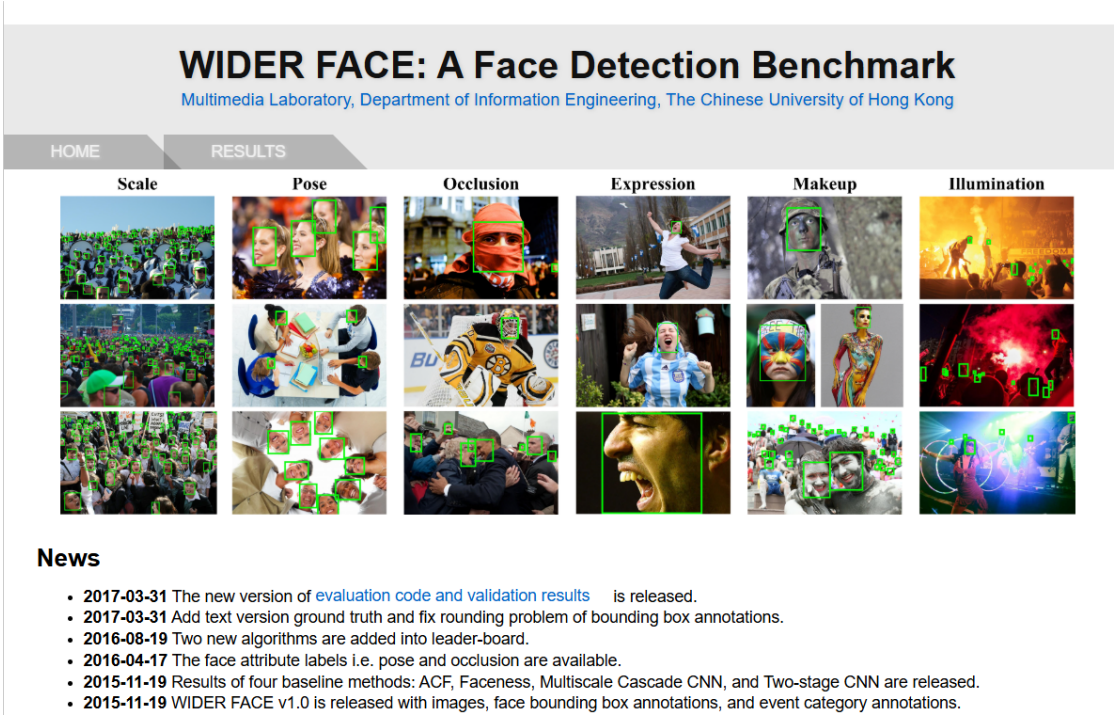
Hiện nay, repository của insightface cung cấp đầy đủ các mô hình đã được huấn luyện sẵn cũng như mã nguồn cho toàn bộ quá trình huấn luyện và suy luận của bước Face Detection. Nhờ đó, người dùng có thể dễ dàng sử dụng trực tiếp các mô hình có sẵn hoặc tùy chỉnh và huấn luyện lại mô hình theo nhu cầu của mình.

II.3. Chuẩn bị dữ liệu

Trong phần này, chúng ta tiến hành chuẩn bị dữ liệu huấn luyện cho bài toán Face Detection. Bộ dữ liệu được sử dụng là WIDER FACE, một trong những bộ dữ liệu phổ biến trong nghiên cứu phát hiện khuôn mặt với nhiều trường hợp khó như khuôn mặt nhỏ, bị che khuất hoặc có góc nhìn phức tạp.

II.3.1. Tải bộ dữ liệu

Truy cập trang web chính thức của bộ dữ liệu:



Hình 5: `http://shuoyang1213.me/WIDERFACE/`

Tiến hành tải về ba thành phần sau:

- WIDER Face Training Images
- WIDER Face Validation Images
- Face Annotations

Sau khi tải về, thực hiện giải nén các tệp. Cấu trúc thư mục sau khi giải nén có dạng:

Cấu trúc thư mục WIDERFACE

```
WIDERFACE/
├── WIDER_train/
│   └── images/
├── WIDER_val/
│   └── images/
└── wider_face_split/
    ├── wider_face_train_bbx_gt.txt
    └── wider_face_val_bbx_gt.txt
```

Trong đó:

- `WIDER_train/images`: chứa ảnh dùng cho huấn luyện
- `WIDER_val/images`: chứa ảnh dùng cho validation
- `wider_face_train_bbx_gt.txt`: file annotation cho tập train
- `wider_face_val_bbx_gt.txt`: file annotation cho tập validation

II.3.2. Định dạng annotation của WIDER FACE

Bộ dữ liệu WIDER FACE sử dụng định dạng annotation dạng COCO. Mỗi ảnh sẽ được mô tả bởi ba phần thông tin chính (Thông tin chi tiết được mô tả ở file readme.txt trong folder `wider_face_split`):

1. Đường dẫn ảnh
2. Số lượng khuôn mặt trong ảnh
3. Thông tin bounding box của từng khuôn mặt

Ví dụ một đoạn trong file `wider_face_train_bbx_gt.txt`:

```
0--Parade/0_Parade_marchingband_1_849.jpg
1
449 330 122 149 0 0 0 0 0 0
```

Ý nghĩa của từng dòng như sau:

- Dòng 1: đường dẫn tương đối của ảnh

  `0--Parade/0_Parade_marchingband_1_849.jpg`

  Ảnh này nằm trong thư mục:

  `WIDER_train/images/0--Parade/`

- Dòng 2: số lượng khuôn mặt trong ảnh

  `1`

  Tức là ảnh này có một khuôn mặt được gán nhãn.

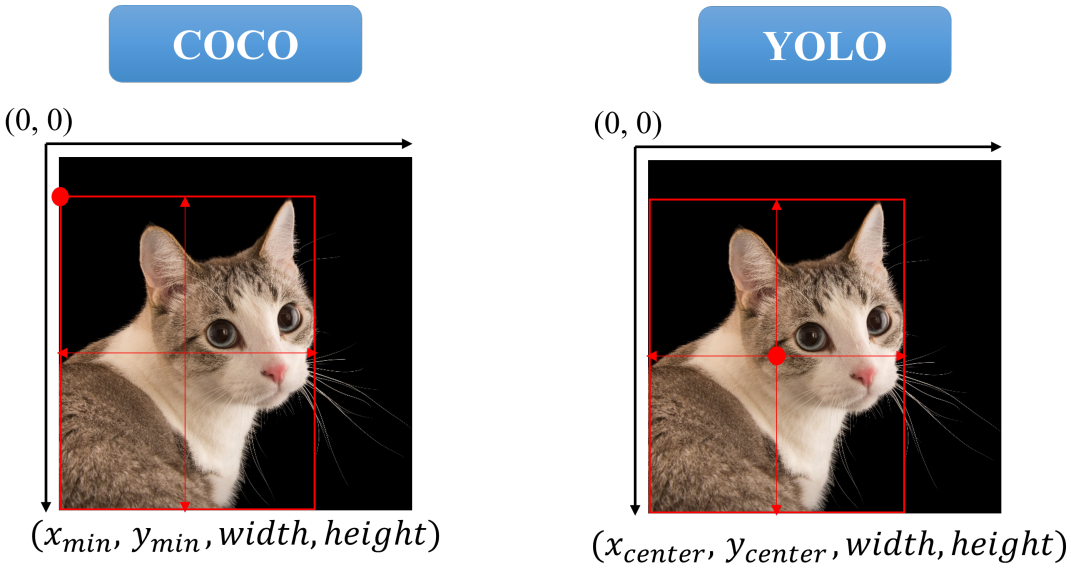
- Dòng 3: thông tin bounding box

  `449 330 122 149 0 0 0 0 0 0`

  - Trong đó bốn giá trị đầu tiên là thông tin bounding box: x, y, w, h
  - Các giá trị phía sau lần lượt biểu thị các thuộc tính bổ sung như: blur, expression, illumination, invalid, occlusion, pose.

Trong quá trình huấn luyện mô hình phát hiện khuôn mặt cơ bản, các thuộc tính bổ sung này thường không được sử dụng.

Để tiện lợi cho quá trình training, chúng ta sẽ chuyển format của dataset từ COCO sang YOLO (như hình 6).



Hình 6: So sánh định dạng của COCO và YOLO

II.3.3. Chuyển đổi sang định dạng YOLO

Nhiều framework huấn luyện object detection thường sử dụng định dạng YOLO. Trong định dạng này, mỗi bounding box được biểu diễn bởi:

$$\text{class_id} \quad x_{center} \quad y_{center} \quad width \quad height$$

Trong đó:

- x_{center}, y_{center} là tọa độ tâm của bounding box
- $width, height$ là kích thước bounding box
- tất cả các giá trị đều được chuẩn hóa về khoảng $[0, 1]$

Quá trình chuyển đổi từ định dạng WIDER FACE sang YOLO được thực hiện như sau:

$$x_{center} = \frac{x + w/2}{W}, \quad y_{center} = \frac{y + h/2}{H}, \quad width = \frac{w}{W}, \quad height = \frac{h}{H}$$

Trong đó W, H là chiều rộng và chiều cao của ảnh.

Để thực hiện chuyển đổi tự động, sử dụng đoạn mã Python sau:

Convert WIDER FACE Train Annotation to YOLO format

```python
import os
import cv2

ann_file = "wider_face_split/wider_face_train_bbx_gt.txt"
image_dir = "WIDER_train/images"
label_dir = "labels"

os.makedirs(label_dir, exist_ok=True)

with open(ann_file) as f:
    lines = f.readlines()

i = 0
while i < len(lines):
    img_path = lines[i].strip()
    i += 1
    num_faces = int(lines[i].strip())
    i += 1

    img_full_path = os.path.join(image_dir, img_path)

    if not os.path.exists(img_full_path):
        print(f"Missing image: {img_full_path} - Skipping")
        i += 1 if num_faces == 0 else num_faces
        continue

    img = cv2.imread(img_full_path)

    if img is None:
        print(f"Could not read: {img_full_path} - Skipping")
        i += 1 if num_faces == 0 else num_faces
        continue

    h, w = img.shape[:2]

    label_path = os.path.join(label_dir, img_path.replace(".jpg", ".txt"))
    os.makedirs(os.path.dirname(label_path), exist_ok=True)

    with open(label_path, "w") as label_file:
        if num_faces == 0:
            i += 1
        else:
            for _ in range(num_faces):
                coords = lines[i].split()
                x, y, bw, bh = map(int, coords[:4])
```

```
                i += 1

                if bw <= 0 or bh <= 0:
                    continue

                x_center = (x + bw / 2) / w
                y_center = (y + bh / 2) / h
                bw /= w
                bh /= h

                label_file.write(f"0 {x_center:.6f} {y_center:.6f} {bw:.6f} {bh:.6f}\n"
                                 )

print("Conversion to YOLO format complete!")
```

Đối với tập validation, chỉ cần thay đổi biến `ann_file` thành `wider_face_val_bbx_gt.txt` và `image_dỉ` thành `"WIDER_valimages"`

II.3.4. Tổ chức lại cấu trúc dataset

Sau khi chuyển đổi annotation, cần tổ chức lại dữ liệu theo cấu trúc chuẩn để thuận tiện cho việc huấn luyện và quản lý dataset.

Cấu trúc thư mục Dataset

```
dataset/
├── train/
│   ├── images/
│   └── labels/
└── val/
    ├── images/
    └── labels/
```

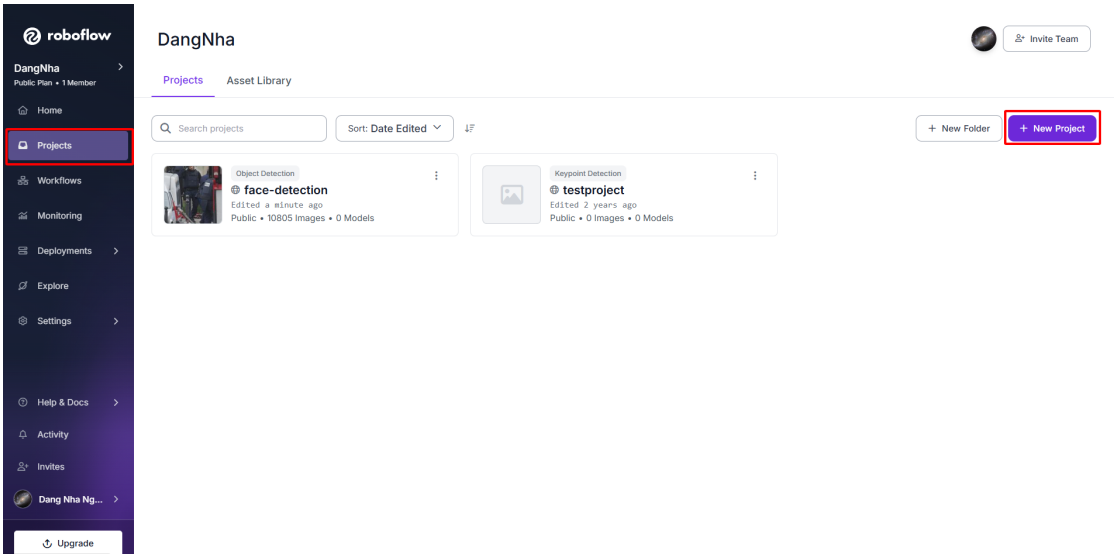
Trong đó:

- `images`: chứa ảnh gốc
- `labels`: chứa annotation theo định dạng YOLO

Mỗi ảnh sẽ có một file annotation tương ứng cùng tên.

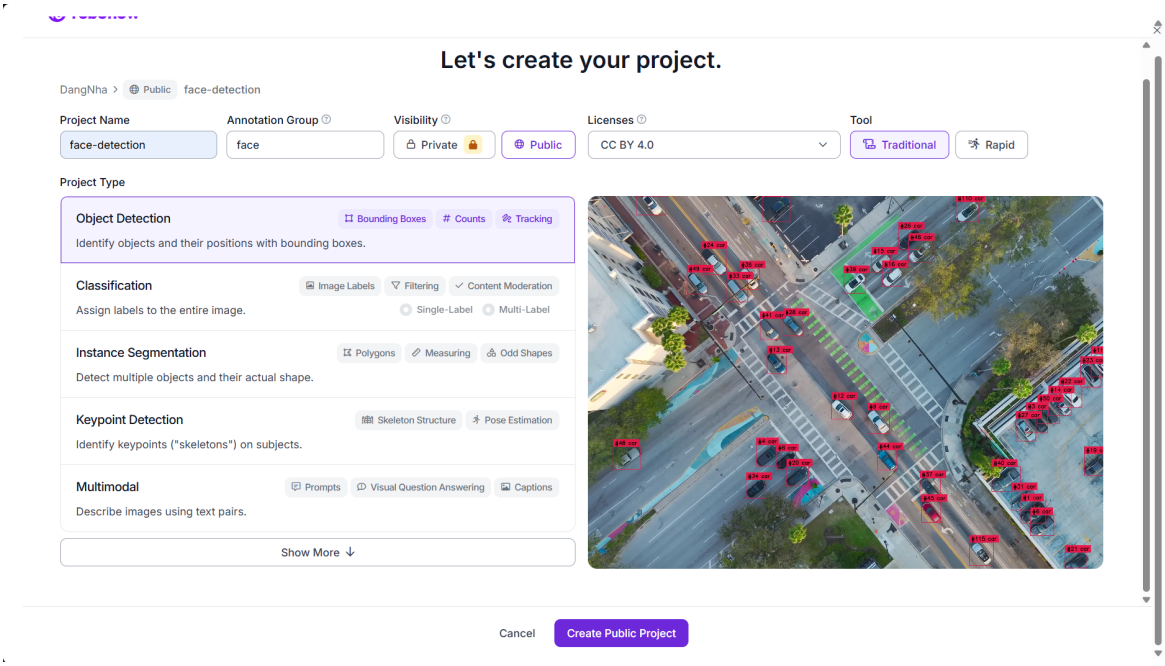
II.3.5. Tải dataset lên Roboflow

Sau khi chuẩn bị dữ liệu, tiến hành tải dataset lên Roboflow để quản lý và tạo phiên bản dataset.
Bước 1: Thực hiện đăng nhập và tạo project trên Roboflow:



Hình 7: Sau khi đăng nhập, chọn Project, sau đó chọn New Project để khởi tạo Project trên Roboflow

Sau đó, lần lượt đặt tên cho project, cài đặt annotation class là "face", lựa chọn public hoặc private cho project, chọn chế độ "Traditional"và chọn loại project Object Detection (tương tự hình 8).



Hình 8: Thực hiện cài đặt project

Bước 2: Vì dataset của chúng ta khá lớn, việc dùng Roboflow UI upload toàn bộ dataset lên sẽ dẫn tới đơ máy hoặc tốn cực lớn tài nguyên. Vậy nên Roboflow đã cung cấp công cụ Roboflow CLI để thực hiện các thao tác liên quan. Chúng ta sẽ thực hiện cài đặt thư viện Roboflow:

Install Roboflow

```
pip install roboflow
```

Tiếp theo thực hiện đăng nhập:

Login Roboflow

```
roboflow login
```

Hệ thống sẽ yêu cầu token đăng nhập. Truy cập: `https://app.roboflow.com/auth-cli`

Tiếp đó, sao chép token và dán vào terminal. Sau khi đăng nhập thành công, lấy các thông tin:

- Workspace ID
- Project ID

Cuối cùng thực hiện lệnh upload dataset:

Roboflow Upload Dataset

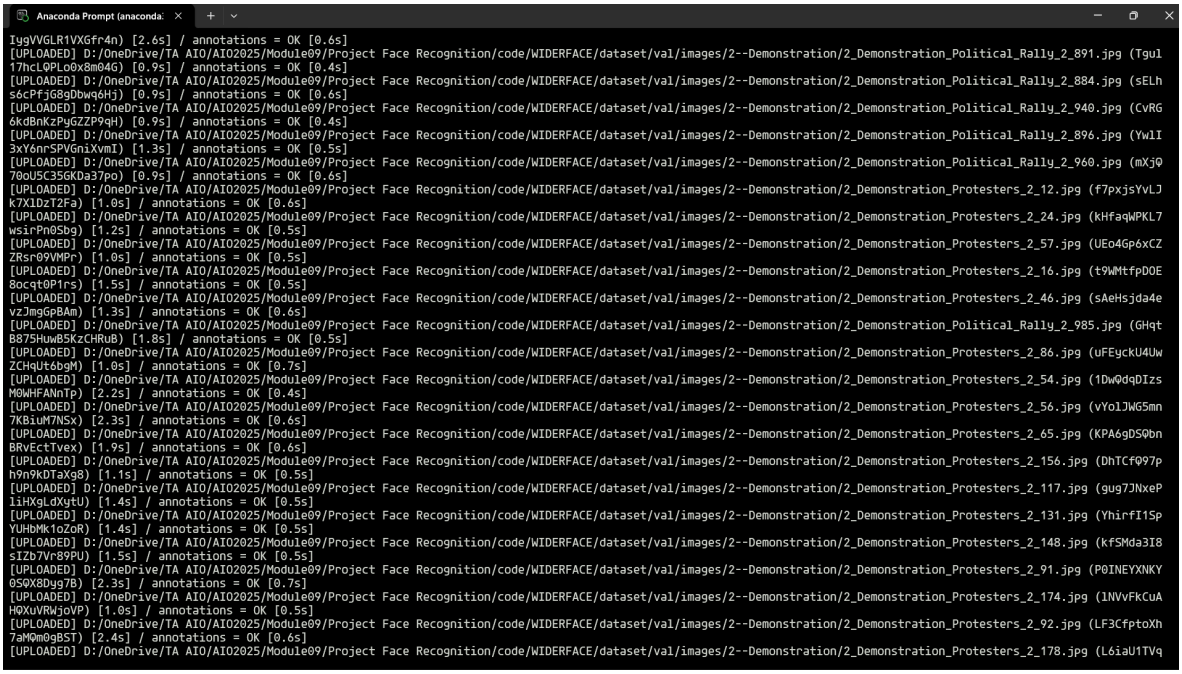
```
roboflow import -w WORKSPACE_ID -p PROJECT_ID /path/to/dataset/folder
```

Ví dụ:

Roboflow Upload Dataset

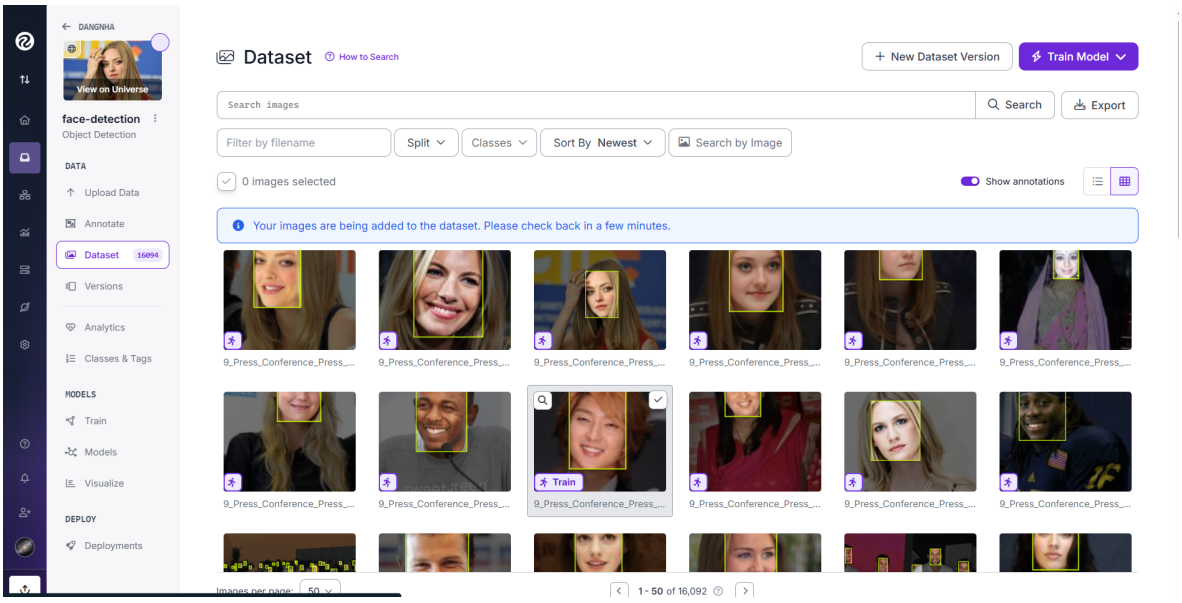
```
roboflow import -w dangnha -p face-detection-besj6
"D:\OneDrive\TA AIO\AIO2025\Module09\Project Face Recognition\code\WIDERFACE\dataset"
```

Sau khi hoàn tất, màn hình hiển thị sẽ như hình 9, toàn bộ ảnh và annotation sẽ được tải lên project Roboflow.



Hình 9: Màn hình hiển thị quá trình upload dataset bằng Roboflow CLI

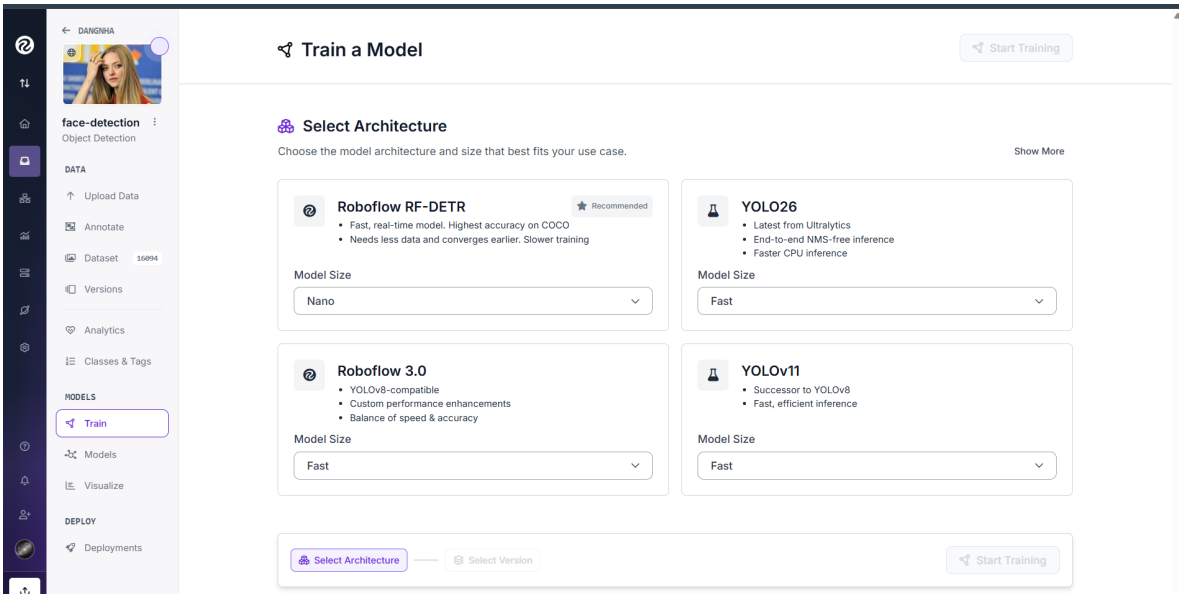
Chúng ta có thể tiếp tục tạo các phiên bản dataset, thực hiện augmentation hoặc export dữ liệu phục vụ cho bước huấn luyện mô hình. Ngoài ra, trong quá trình thực hiện tải xuống, upload dataset, chúng ta hoàn toàn có thể thêm folder dữ liệu cá nhân nhằm align mô hình gần hơn với bài toán mà chúng ta sử dụng (ví dụ: thêm tập ảnh employees của công ty vào tập dữ liệu để mô hình học qua các khuôn mặt cần detect)



Hình 10: Kết quả sau khi upload xong toàn bộ dataset.

II.4. Huấn luyện mô hình

Để thực hiện huấn luyện mô hình Face Detection, chúng ta có thể chọn bất kỳ mô hình Detection nào phù hợp. Ở project này, 2 lựa chọn được đưa ra là YOLO-based hoặc SCRFD của repository insightface. Với SCRFD, mọi người có thể tham khảo cách training của repo ở file README.md. Với lựa chọn YOLO, chúng ta có thể thực hiện training ngay trên ROBOFLOW ở phần MODELS > Train:



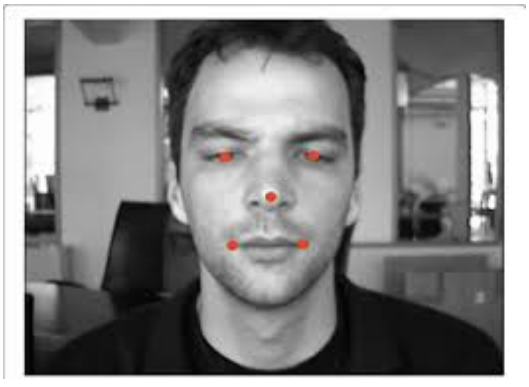
Hình 11: Training model với Roboflow

II.5. Face Alignment

Sau bước phát hiện khuôn mặt bằng mô hình SCRFD (nếu chúng ta dùng SCRFD thay vì YOLO), hệ thống thu được hai thành phần quan trọng: *bounding box* của khuôn mặt và tập hợp các *facial landmarks*.
Trong trường hợp này, SCRFD cung cấp 5 điểm mốc đặc trưng của khuôn mặt bao gồm: hai mắt, mũi, và hai khóe miệng.
Các điểm mốc này được biểu diễn dưới dạng:

$$\mathbf{p}_i = (x_i, y_i), \quad i \in \{1, 2, 3, 4, 5\}$$

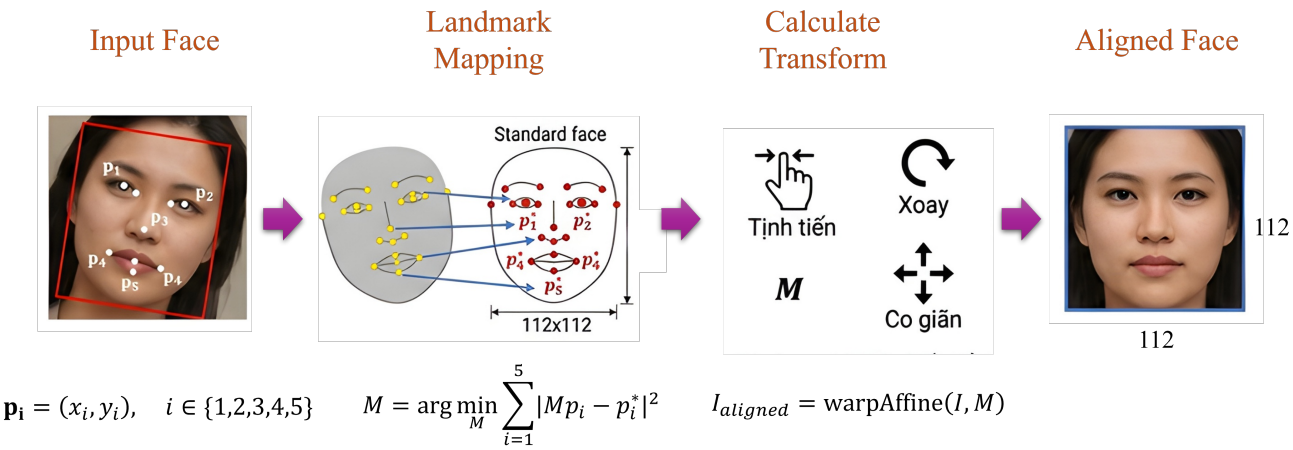
trong đó mỗi điểm tương ứng với một vị trí đặc trưng trên khuôn mặt.



Hình 12: Ví dụ về Landmark với 5 điểm mốc đặc trưng (Reference)

Tuy nhiên, các khuôn mặt được phát hiện trong ảnh đầu vào thường có sự khác biệt về góc quay, tỉ lệ, hoặc vị trí trong khung hình. Để đảm bảo tính nhất quán của biểu diễn đặc trưng, cần thực hiện bước **Face Alignment**. Mục tiêu của bước này là chuẩn hóa khuôn mặt về một tư thế chuẩn trước khi đưa vào mô hình nhận dạng.

Quá trình căn chỉnh được thực hiện bằng phép biến đổi *Similarity Transform*, bao gồm ba thành phần: tịnh tiến (translation), xoay (rotation), và co giãn (scaling). Phép biến đổi này ánh xạ các điểm mốc phát hiện được sang một bộ điểm mốc chuẩn được định nghĩa trước.



Hình 13: Quá trình thực hiện Face Aligment

Gọi $\mathbf{P} = \{p_i\}_{i=1}^5$ là tập điểm mốc phát hiện được và $\mathbf{P}^* = \{p_i^*\}_{i=1}^5$ là tập điểm mốc chuẩn. Ma trận biến đổi affine M được ước lượng bằng cách tối thiểu hóa sai số giữa hai tập điểm:

$$M = \arg\min_M \sum_{i=1}^{5} \|Mp_i - p_i^*\|^2$$

Sau khi tính được ma trận biến đổi M, ảnh khuôn mặt được biến đổi hình học thông qua phép nội suy affine:

$$I_{aligned} = \text{warpAffine}(I, M)$$

Kết quả thu được là khuôn mặt đã được căn chỉnh với kích thước cố định 112×112 pixels, trong đó các đặc điểm khuôn mặt (đặc biệt là hai mắt) được đặt ở vị trí chuẩn. Bước chuẩn hóa này giúp giảm sự biến thiên do góc nhìn và vị trí, từ đó cải thiện độ ổn định của vector đặc trưng khi trích xuất embedding bằng mô hình ArcFace.

Triển khai trong API hệ thống: Trong hệ thống được triển khai, bước Face Alignment được thực hiện nội bộ trong phương thức `get_embedding()` của lớp `ArcFace`. Sau khi mô hình SCRFD phát hiện khuôn mặt và trả về tập keypoints, các điểm này được truyền trực tiếp vào hàm trích xuất embedding:

Face Aligment

```
embedding = state.recognizer.get_embedding(frame, kps)
```

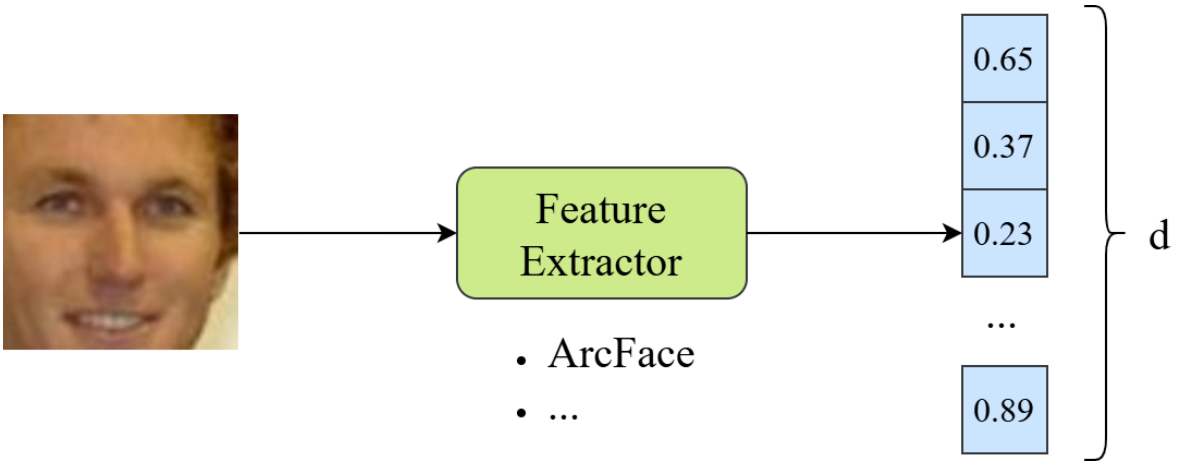
Bên trong phương thức này, ảnh đầu vào cùng với các keypoints được sử dụng để thực hiện phép biến đổi căn chỉnh khuôn mặt trước khi được chuẩn hóa kích thước và đưa vào mô hình ArcFace để trích xuất vector đặc trưng. Cách triển khai này giúp tách biệt rõ ràng giữa bước phát hiện khuôn mặt và bước chuẩn hóa khuôn mặt, đồng thời đảm bảo mọi embedding được tính toán từ các khuôn mặt đã được căn chỉnh theo cùng một chuẩn hình học.

III. Feature Extraction

III.1. Giới thiệu về Feature Extraction

Sau khi đã trích xuất được khuôn mặt, việc tiếp theo chính là Feature Extraction (trích xuất đặc trưng của khuôn mặt). Ở bước này, ta sẽ sử dụng một mô hình Deep Learning để ánh xạ hình ảnh khuôn mặt sang một vector đặc trưng trong không gian nhiều chiều.

Vector đặc trưng này có khả năng biểu diễn các đặc điểm quan trọng của khuôn mặt, sao cho các khuôn mặt của cùng một người sẽ có vector đặc trưng gần nhau, trong khi khuôn mặt của những người khác nhau sẽ có khoảng cách lớn hơn trong không gian đặc trưng.



Hình 14: Minh họa đầu vào và đầu ra bước Feature Extraction

Cụ thể, bước này có đầu vào và đầu ra như sau:

- **Input:** Ảnh khuôn mặt đã được cắt và căn chỉnh, có kích thước $x \in \mathbb{R}^{C \times H \times W}$.
- **Output:** Vector đặc trưng $f \in \mathbb{R}^d$ biểu diễn khuôn mặt trong không gian đặc trưng.

Trong đó:

- C là số kênh màu của ảnh.

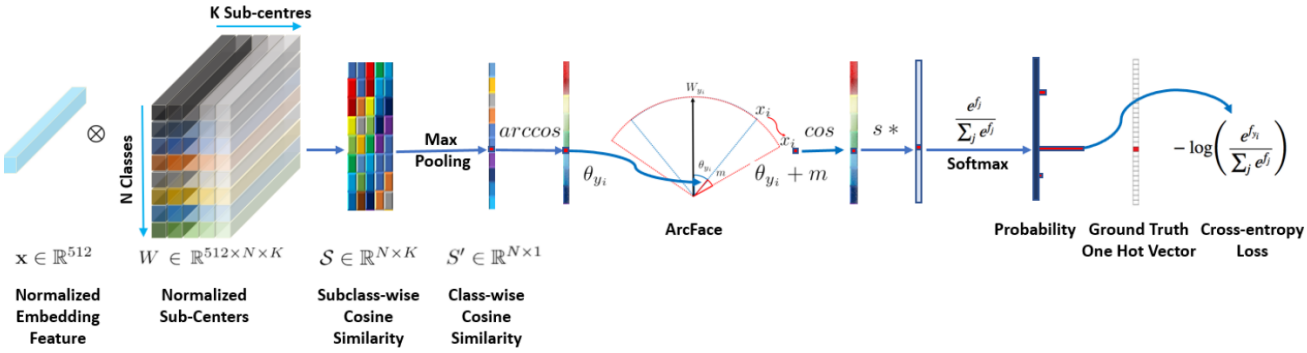
- H và W lần lượt là chiều cao và chiều rộng của ảnh khuôn mặt đầu vào.
- d là số chiều của vector đặc trưng

Tương tự như bước Face Detection, bước Feature Extraction cũng đòi hỏi mô hình phải đạt được độ chính xác cao đồng thời đảm bảo thời gian suy luận đủ nhanh để có thể sử dụng trong các ứng dụng thực tế. Một trong những phương pháp phổ biến hiện nay cho bước này là ArcFace, một mô hình học biểu diễn khuôn mặt hiệu quả đã đạt được nhiều kết quả tốt trên các bộ dữ liệu benchmark.

III.2. Phương pháp ArcFace

Bên cạnh việc xây dựng kiến trúc mô hình, việc lựa chọn loss function cũng đóng vai trò quan trọng đối với hiệu suất của mô hình. Một thách thức của bài toán Feature Extraction từ khuôn mặt là sự tương đồng cao giữa các điểm dữ liệu, đặc biệt là giữa các khuôn mặt của những người khác nhau. Điều này đòi hỏi phương pháp không chỉ cần trích xuất được các đặc trưng có ý nghĩa mà còn phải đảm bảo rằng các đặc trưng này có khả năng phân biệt tốt giữa các danh tính khác nhau.

ArcFace (Additive Angular Margin Loss for Deep Face Recognition) là một phương pháp học biểu diễn khuôn mặt hiệu quả được đề xuất nhằm cải thiện khả năng phân biệt giữa các danh tính khác nhau trong bài toán Face Recognition. Thay vì chỉ tối ưu hóa khoảng cách giữa các vector đặc trưng trong không gian Euclidean, ArcFace đưa ra một hàm mất mát mới dựa trên khoảng cách góc trong không gian đặc trưng.



Hình 15: Hình ảnh minh họa cho ArcFace

Cụ thể, hàm loss của ArcFace được định nghĩa như sau:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{e^{s(\cos(\theta_{y_i}+m))}}{e^{s(\cos(\theta_{y_i}+m))}+\sum_{j\neq y_i}e^{s\cos\theta_j}}$$

Trong đó:

- N là số lượng mẫu trong một batch.
- θ_j là góc giữa vector đặc trưng của mẫu đầu vào và vector trọng số của lớp j.

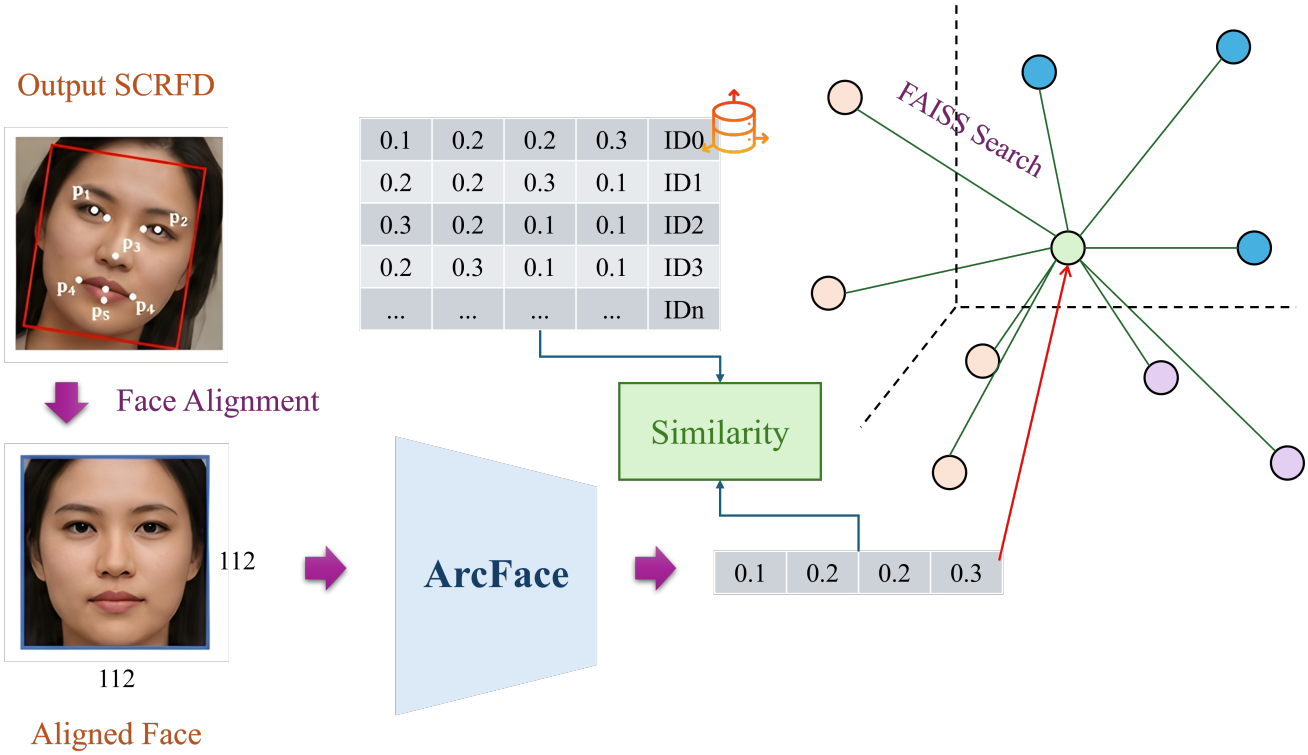
- θ_{y_i} là góc tương ứng với lớp đúng của mẫu i.
- m là *angular margin* được thêm vào lớp đúng nhằm tăng khả năng phân biệt giữa các lớp.
- s là hệ số tỉ lệ (*scale factor*) được sử dụng để ổn định quá trình huấn luyện.

Việc thêm *angular margin* vào lớp đúng giúp tăng khoảng cách giữa các lớp khác nhau trong không gian đặc trưng, đồng thời làm cho các vector đặc trưng của cùng một danh tính trở nên tập trung hơn.

Trong một số biến thể của ArcFace, mỗi lớp có thể được biểu diễn bởi nhiều *sub-centers*. Độ tương đồng cosine giữa vector đặc trưng và các sub-centers sẽ được tính toán, sau đó giá trị lớn nhất được chọn làm độ tương đồng của lớp trước khi đưa vào hàm loss như trên Hình15 .

Chuẩn bị dữ liệu và huấn luyện mô hình Ở bước này, chúng ta sử dụng mô hình arcface cho tác vụ Face Extraction, vậy nên bước download dataset, hay training model, chúng ta có thể tham khảo file README.md, chi tiết về quá trình cài đặt và huấn luyện cũng như các mô hình đã được huấn luyện sẵn đề có sẵn ở repository insightface.

IV. Feature Matching



Hình 16: Quá trình thực hiện Matching tìm danh tính khuôn mặt

Sau khi khuôn mặt được căn chỉnh (Face Alignment), ảnh khuôn mặt chuẩn hóa sẽ được đưa vào mô hình trích xuất đặc trưng ArcFace để tạo ra một vector đặc trưng (embedding). Embedding

này là một vector biểu diễn đặc trưng nhận dạng của khuôn mặt trong không gian latent. Cho một khuôn mặt đầu vào, mô hình ArcFace sinh ra vector đặc trưng:

$$\mathbf{f} \in \mathbb{R}^d$$

Trong quá trình xây dựng cơ sở dữ liệu khuôn mặt, mỗi ảnh khuôn mặt đã biết cũng được chuyển thành một embedding tương ứng và lưu trữ trong cơ sở dữ liệu vector. Trong hệ thống này, các embedding được lưu trữ và lập chỉ mục bằng thư viện **FAISS (Facebook AI Similarity Search)**, một thư viện tối ưu cho việc tìm kiếm vector trong một database có số lượng vector lớn.

Để xác định danh tính của khuôn mặt đầu vào, hệ thống thực hiện so sánh embedding $\mathbf{f}$ với toàn bộ các embedding đã lưu $\mathbf{f}_i$ trong cơ sở dữ liệu. Độ tương đồng giữa hai vector được đo bằng **Cosine Similarity**:

$$\text{sim}(\mathbf{f}, \mathbf{f}_i) = \frac{\mathbf{f} \cdot \mathbf{f}_i}{\|\mathbf{f}\| \, \|\mathbf{f}_i\|}$$

trong đó $\mathbf{f} \cdot \mathbf{f}_i$ là tích vô hướng của hai vector, và $\|\mathbf{f}\|$ là chuẩn Euclid của vector.

Giá trị cosine similarity nằm trong khoảng $[-1, 1]$, trong đó giá trị càng gần 1 cho thấy hai vector càng giống nhau.

Trong quá trình truy vấn, FAISS thực hiện tìm kiếm *nearest neighbor* để xác định embedding trong cơ sở dữ liệu có độ tương đồng cao nhất với embedding đầu vào:

$$i^* = \arg\max_i \ \text{sim}(\mathbf{f}, \mathbf{f}_i)$$

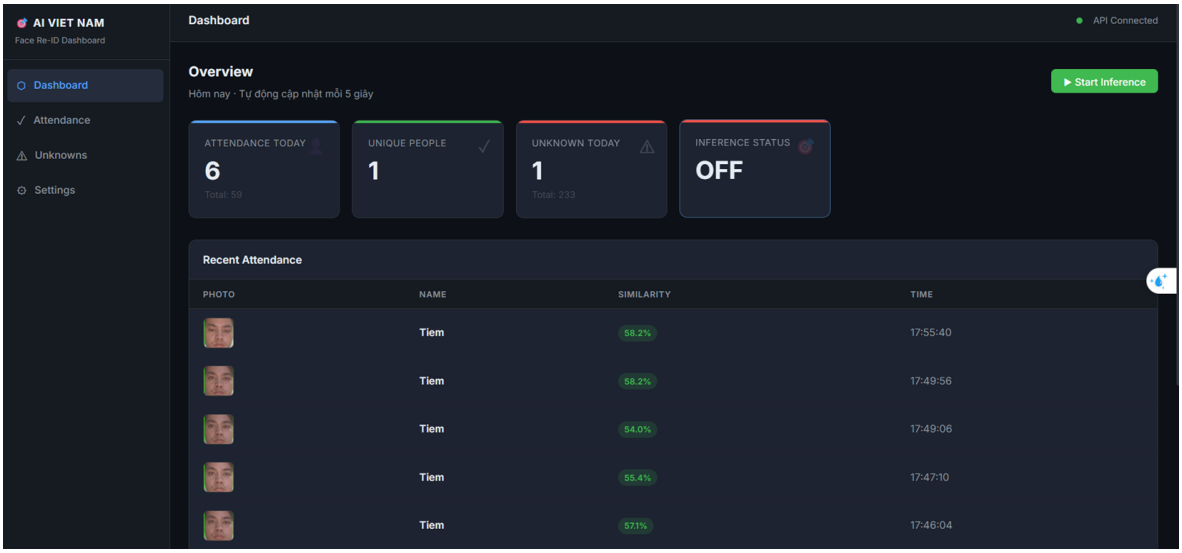
Nếu giá trị tương đồng lớn hơn một ngưỡng xác định trước τ, hệ thống sẽ gán khuôn mặt đầu vào với danh tính tương ứng trong cơ sở dữ liệu. Ngược lại, khuôn mặt sẽ được phân loại là *unknown*.

V. Triển khai hệ thống

V.1. Tổng quan về luồng hoạt động

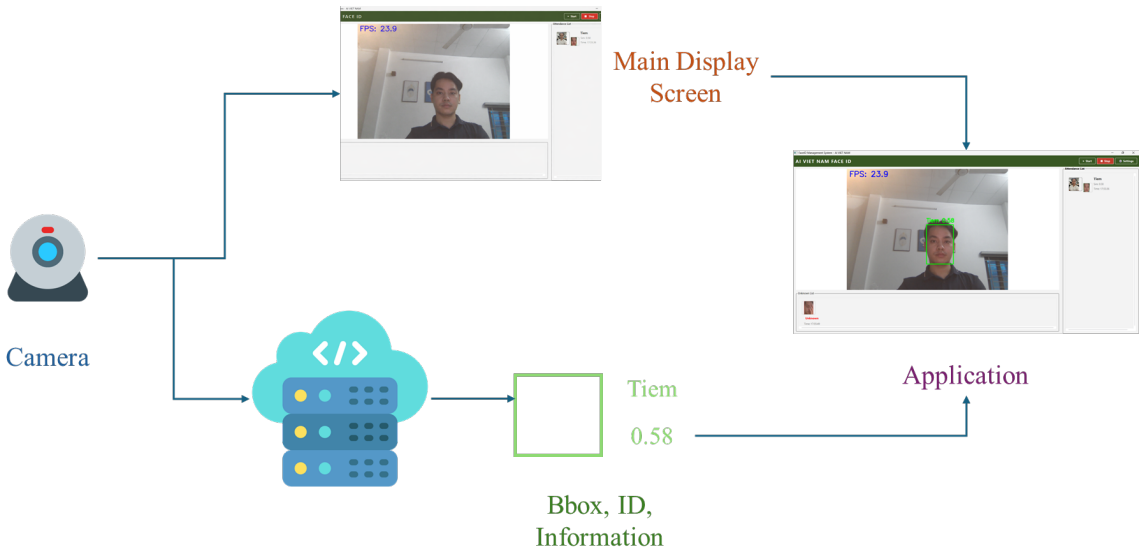
Hình 2 minh họa kiến trúc tổng quát của hệ thống. Hệ thống bao gồm ba thành phần chính: giao diện người dùng (GUI), dịch vụ nhận dạng khuôn mặt phía máy chủ (API backend) và lớp quản lý cơ sở dữ liệu. Các khung hình video được thu thập từ GUI (PyQt6) được truyền tới API phía máy chủ (FastAPI) thông qua kết nối WebSocket nhằm hỗ trợ xử lý luồng dữ liệu theo thời gian thực. Tại phía máy chủ, hệ thống trước tiên phát hiện khuôn mặt bằng mô hình SCRFD, sau đó thực hiện bước Face Alignment để chuẩn hóa các điểm mốc (landmarks) trên khuôn mặt. Các ảnh khuôn mặt đã được căn chỉnh tiếp tục được đưa vào mô hình ArcFace để trích xuất đặc trưng dưới dạng vector embedding. Các vector này được so khớp với các embedding đã lưu

trữ trong cơ sở dữ liệu vector dựa trên FAISS bằng cách sử dụng độ đo cosine similarity nhằm tìm ra danh tính có độ tương đồng cao nhất. Kết quả nhận dạng, bao gồm các bounding box và danh tính dự đoán, được trả về GUI để trực quan hóa, trong khi các siêu dữ liệu liên quan có thể được lưu trữ và quản lý trong cơ sở dữ liệu PostgreSQL. Ngoài ra, hệ thống còn có bảng điều khiển được xây dựng bằng React giao tiếp với backend thông qua các REST API để hỗ trợ giám sát và quản lý dữ liệu nhận dạng.



Hình 17: Giao diện bảng theo dõi hệ thống

V.2. Xử lý camera



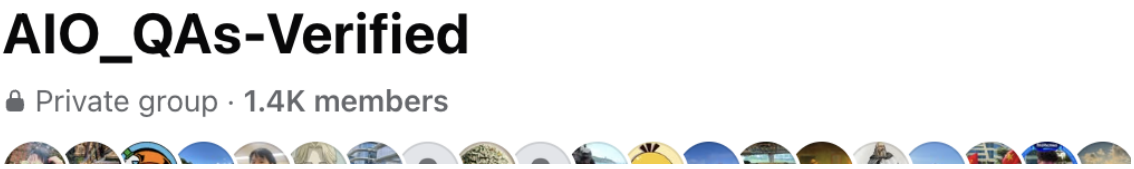
Hình 18: Luồng xử lý camera cho hệ thống

Hệ thống chia luồng xử lý video thành hai nhánh riêng biệt nhằm tối ưu hóa hiệu năng của bài toán nhận dạng khuôn mặt theo thời gian thực. Nhánh thứ nhất truyền trực tiếp luồng video

từ camera tới giao diện chính để hiển thị liên tục cho người dùng, đảm bảo tốc độ khung hình (FPS) ổn định và giảm độ trễ hiển thị. Nhánh thứ hai gửi các khung hình tới máy chủ để thực hiện các bước xử lý nhận dạng khuôn mặt, bao gồm phát hiện khuôn mặt, trích xuất đặc trưng và so khớp danh tính. Kết quả nhận dạng, bao gồm bounding box, danh tính dự đoán và độ tương đồng, sau đó được trả về ứng dụng và chồng lên luồng video hiển thị. Việc tách hai luồng xử lý này giúp giảm tải cho quá trình suy luận mô hình, tránh làm gián đoạn quá trình hiển thị video và đảm bảo hệ thống có thể duy trì khả năng xử lý theo thời gian thực.

Phụ lục

1. **Hint:** Repository của dự án mọi người có thể tải tại đây.
2. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng 3-4 tiếng.



Hình 1: Hình ảnh group facebook AIO Q&A.

3. **Các hướng mở rộng**
   - Thu thập thêm dữ liệu khuôn mặt trong các điều kiện môi trường khó (ánh sáng yếu, góc nhìn nghiêng, che khuất) để huấn luyện lại các mô hình face detection và feature extraction.
   - Tích hợp cơ chế face anti-spoofing để phát hiện các hành vi gian lận, chẳng hạn như sử dụng ảnh hoặc video nhằm giả mạo danh tính khi thực hiện điểm danh.
   - Mở rộng hệ thống để hỗ trợ xử lý nhiều camera hoặc nhiều luồng video đồng thời, thông qua tối ưu hóa mô hình và cải thiện khả năng xử lý song song của backend.
4. **Rubric:**

Face Recognition System - Rubric		
Câu	**Kiến thức**	**Tiêu chí đánh giá**
1	- Hiểu bài toán **Face Detection** và các thách thức trong phát hiện khuôn mặt. - Nắm được nguyên lý hoạt động của mô hình SCRFD và quy trình phát hiện khuôn mặt.	- Triển khai thành công mô hình SCRFD để phát hiện khuôn mặt trong ảnh hoặc video. - Xác định chính xác bounding box của khuôn mặt trong các khung hình.
2	- Hiểu quy trình **Face Alignment** và tầm quan trọng của việc chuẩn hóa landmark khuôn mặt. - Nắm được phương pháp trích xuất đặc trưng khuôn mặt bằng mô hình ArcFace.	- Thực hiện căn chỉnh khuôn mặt dựa trên các điểm landmark. - Trích xuất thành công vector embedding đại diện cho đặc trưng khuôn mặt.
3	- Hiểu nguyên lý **vector similarity search** và cơ chế hoạt động của FAISS. - Nắm được cách sử dụng cosine similarity để so khớp các embedding khuôn mặt.	- Xây dựng cơ sở dữ liệu vector cho các embedding khuôn mặt. - Thực hiện tìm kiếm và xác định danh tính có độ tương đồng cao nhất.
4	- Hiểu kiến trúc hệ thống nhận dạng khuôn mặt thời gian thực. - Nắm được cách tích hợp GUI, API backend và cơ sở dữ liệu trong một hệ thống hoàn chỉnh.	- Triển khai hệ thống nhận dạng khuôn mặt với pipeline hoàn chỉnh. - Hiển thị kết quả nhận dạng (bounding box và danh tính) theo thời gian thực trên giao diện.